

Git Guide — Essential Commands

Basic concepts

Git stores the project history as **commits**; each person works on their own **branch** and only then merges the result into `develop`.

Term	What it is
Repository	The project folder with its full version history
Commit	A "snapshot" of changes, with a message explaining what changed
Branch	A separate line of work; changes there do not affect other branches
Remote (<code>origin</code>)	The copy of the repository on GitHub
<code>master</code>	Stable code, ready to use
<code>develop</code>	Where new features are integrated before going to <code>master</code>

Rule for the `api_demo` project: nobody commits directly to `master` or `develop`. Every change starts on a branch created from `develop`.

Initial setup and clone

Done once per computer: identify yourself to Git and download the `api_demo` repository.

1. Set your name and email (they appear on every commit):

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
```

2. Clone the repository from GitHub:

```
git clone https://github.com/lgbuffa/api_demo.git
cd api_demo
```

3. Switch to the `develop` branch:

```
git checkout develop
```

4. Create the `.env` from the template (`.env` never goes to GitHub):

```
cp .env.example .env
```

On the first `git push`, Windows opens the browser to log in to GitHub; after that the credential is saved.

Workflow: from branch to push

Every task follows the same 6 steps, always starting from an up-to-date `develop`.

1. Go to `develop` and pull the latest changes:

```
git checkout develop
git pull
```

2. Create your branch from it (`-b` creates the branch and switches to it):

```
git checkout -b feature/task-name
```

3. Edit the files in your editor and check what changed:

```
git status
git diff
```

4. Stage the changes for the commit (`.` includes everything in the current folder):

```
git add .
```

5. Record the commit with a clear message:

```
git commit -m "Add severity filter to alarms"
```

6. Push the branch to GitHub (`-u` links the local branch to the remote; next time just `git push`):

```
git push -u origin feature/task-name
```

Branch names: `feature/...` for something new, `fix/...` for a bug fix. Before `git add .`, check in `git status` that no unwanted file (such as `.env`) shows up.

Integrating your work: pull request and merge

A branch gets into `develop` through a **pull request** on GitHub, where someone else reviews it before the merge.

1. On GitHub, open the repository and click **Compare & pull request**.
2. Choose **base:** `develop` and **compare:** `feature/task-name`, describe what changed and create the PR.
3. The reviewer approves and clicks **Merge pull request**.
4. On your computer, update `develop` and delete the local branch that was already merged:

```
git checkout develop
git pull
git branch -d feature/task-name
```

When `develop` is stable, open a PR from `develop` to `master` the same way.

If someone else changed `develop` while you were working, bring those changes into your branch before the PR:

```
git checkout feature/task-name
git merge develop
```

Useful commands for checking and fixing

Command	What it does
<code>git status</code>	Shows which branch you are on and which files changed
<code>git branch -a</code>	Lists local and remote branches
<code>git log --oneline --graph --all</code>	Short commit history, with the branches drawn
<code>git diff</code>	Shows changed lines not yet staged
<code>git restore file.py</code>	Discards uncommitted changes to a file
<code>git restore --staged file.py</code>	Unstages a file, keeping the change
<code>git commit --amend -m "new msg"</code>	Fixes the last commit message (only before pushing)
<code>git stash</code> / <code>git stash pop</code>	Shelves changes temporarily to switch branches, then brings them back
<code>git fetch</code>	Downloads what is new on GitHub without touching your files

Hands-on: api_demo step by step

Starting on `master`, add a print to `fetch_machine_alarms.py`, push it to `develop` and open a pull request from `develop` to `master`.

1. Switch to `develop`:

```
git checkout develop
```

2. Update `develop` with what is on GitHub:

```
git pull
```

3. Open `fetch_machine_alarms.py` and add this line at the end of the file:

```
print(f"Total alarms: {len(alarms)}")
```

Then check the change:

```
git status
git diff
```

4. Stage the change:

```
git add .
```

5. Commit it:

```
git commit -m "Print total number of alarms"
```

6. Push to GitHub:

```
git push
```

7. Create the pull request to `master` :

1. Open https://github.com/lgbuffa/api_demo and go to **Pull requests > New pull request**.
2. Set **base:** `master` and **compare:** `develop` (the arrow points to the base, the destination).
3. Click **Create pull request**, write a title (for example "Print total number of alarms") and confirm.
4. Review the **Files changed** tab, then click **Merge pull request > Confirm merge**.
5. Back in the terminal, update your local `master` :

```
git checkout master
git pull
```